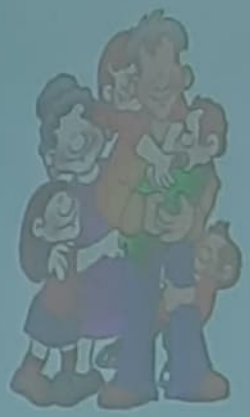
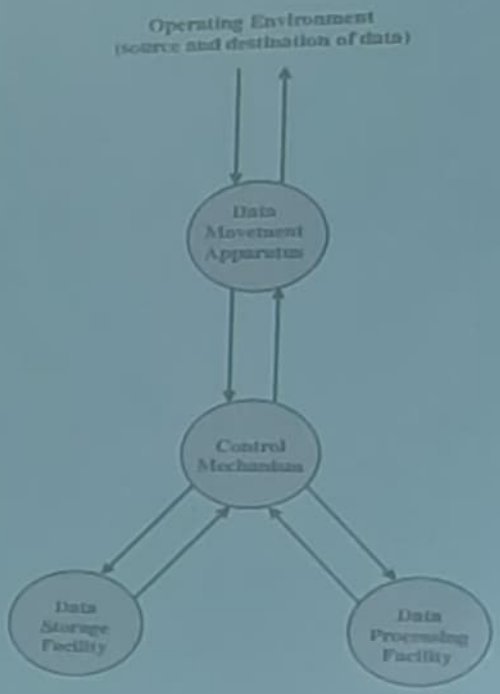
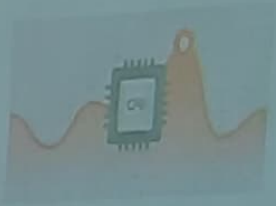
Functional View of a Computer

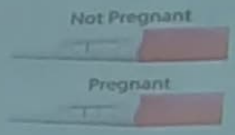




Designing for Performance

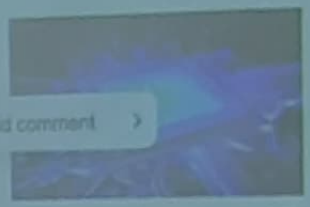
- Year by year, the cost of computer systems continues to drop dramatically, while the performance and capacity of those systems continue to rise equally dramatically.
- Today's laptops have the computing power of an IBM mainframe from 20 or 25 years ago. Thus, we have virtually "free" computer power.
- *Processors are so inexpensive that we now have microprocessors we throw away.*
- *The digital pregnancy test is an example (used once and then thrown away).*
- And this continuing technological revolution has enabled the development of applications of astounding complexity and power.
- For example, desktop applications that require the great power of today's microprocessor-based systems include
  - Image processing
  - Speech recognition
  - Videoconferencing
  - Multimedia authoring
  - Voice and video annotation of files
  - Simulation modeling





What is Performance?

- In co[illegible]
  - Co[illegible]
  - *Computing resources* (what it costs to do it)
- Computational problems translate to *requirements*
- Computing resources interplay and tradeoff



$$\textit{Performance} \sim \frac{1}{\textit{Resources for solution}}$$

Hardware

Time

Energy

... and ultimately

Money

Performance Measures: Execution Time

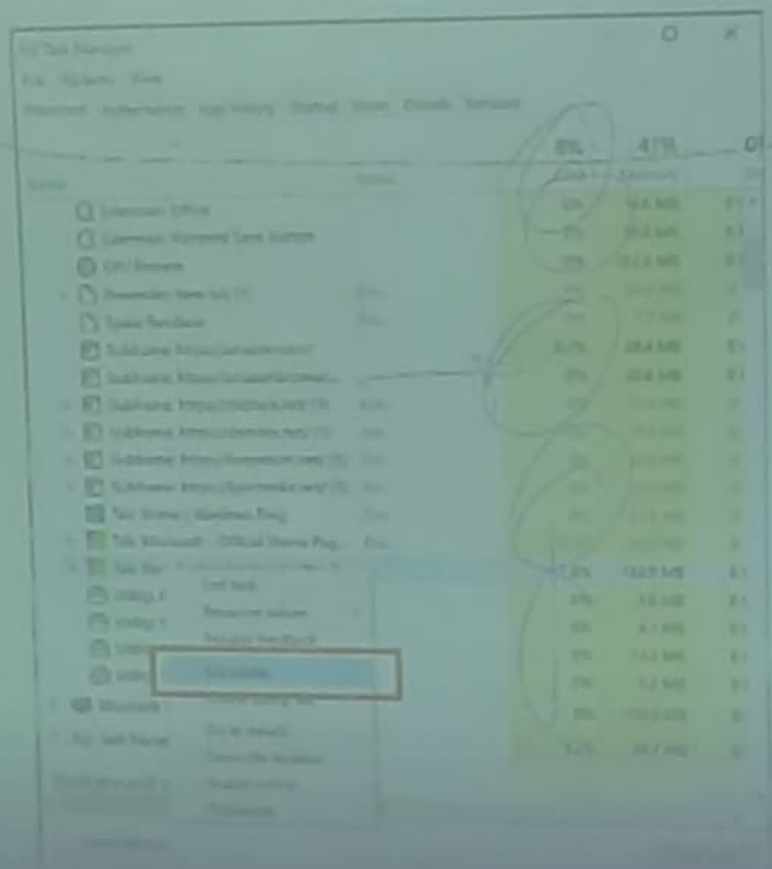
- For a specific program compiled to run on a specific machine (CPU) "A", has the following parameters:
  - The total executed instruction count of the program → IC
  - The average number of cycles per instruction (average CPI) → CPI
  - Clock cycle of machine "A" → CC

Performance Measures: Execution Time

- How can one measure the performance of this machine (CPU) running this program?
  - *Intuitively the machine (or CPU) is said to be faster or has better performance running this program if the total execution time is shorter.*
  - Thus the inverse of the total measured program execution time is a possible performance measure or metric:

$$\text{Performance} = 1 \;/\; \text{ExecutionTime}$$

Computer Performance Instance



11:38 AM · Wed 12 Feb
All tools Edit Export E-Sign

6 Tips for Speeding Up Your Computer Performance

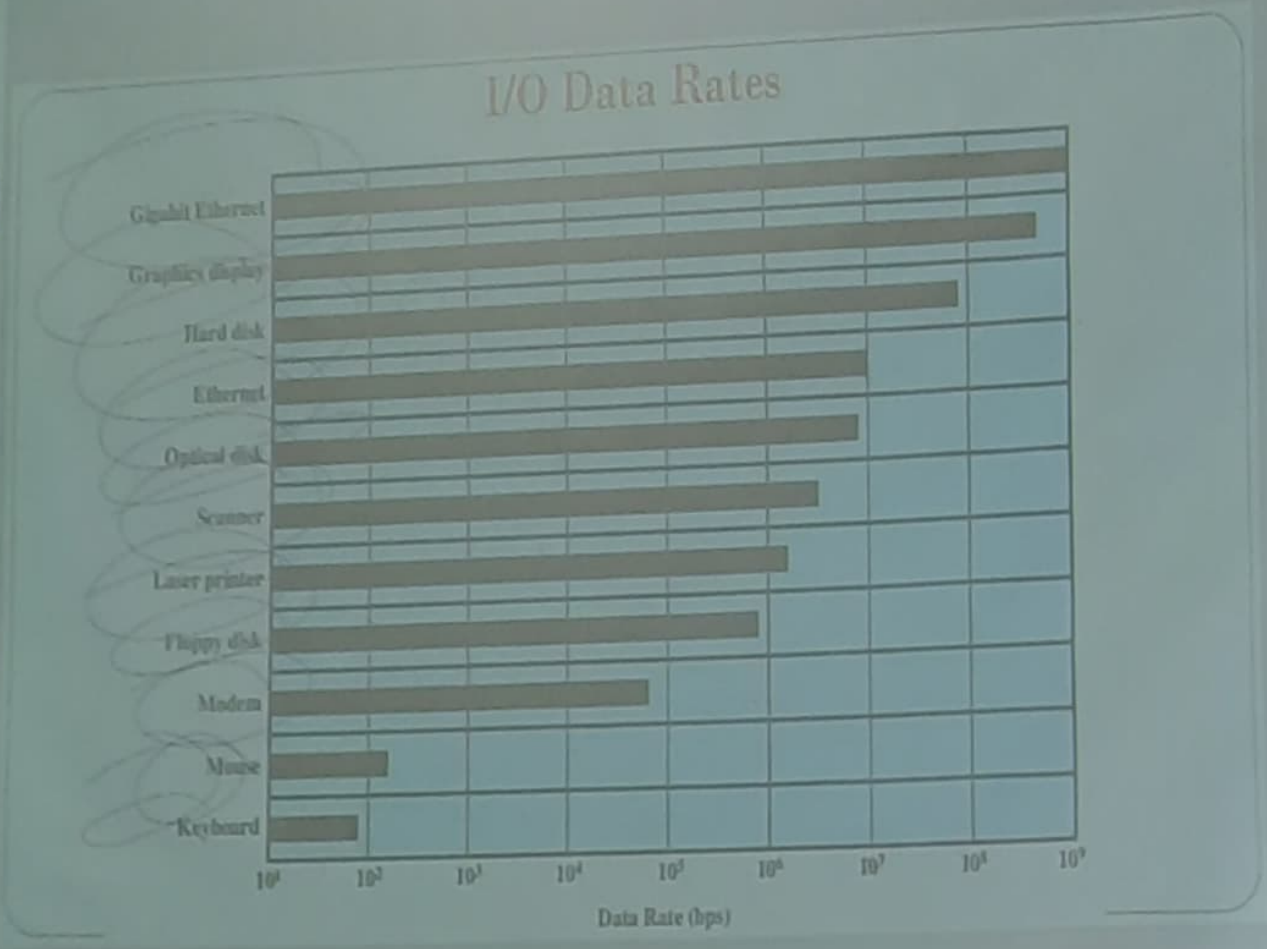
- Clean Up Your Hard Drive
- Disable Startup Programs
- Upgrade Your Hardware
- Use a Disk Cleanup Tool
- Keep Your Computer Cool
- Limit Background Processes

Why Do We Care About Performance?

- *Performance itself is a measure of how well the computational requirements can be satisfied.*
- *We evaluate performance to understand the relationships between requirements and resources*
  - *Decide how to change "solutions" to target objectives*
- Performance measures *reflect decisions about how and how well "solutions" are able to satisfy the computational requirements*

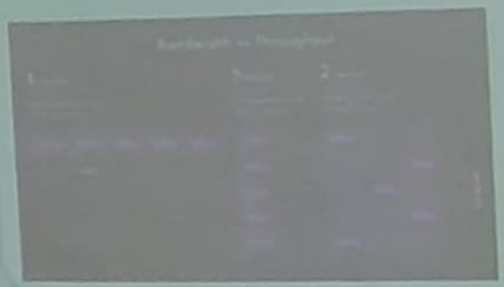
Performance Measures: Evolving Factors

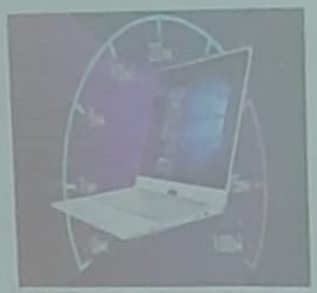
- Designers constantly strive to balance the throughput and processing demands of the processor components, main memory, I/O devices, and the interconnection structures.
- This design must constantly be rethought to cope with two constantly evolving factors:
  - *The rate at which performance is changing in the various technology* areas (processor, buses, memory, peripherals) differs greatly from one type of element to another.
  - *New applications and new peripheral devices* constantly change the nature of the demand on the system in terms of typical instruction profile and the data access patterns.
- Thus, computer design is a constantly evolving art form.


I/O Data Rates
Gigabit Ethernet
Graphics display
Hard disk
Ethernet
Optical disk
Scanner
Laser printer
Floppy disk
Modem
Mouse
Keyboard
10^0
10^2
10^3
10^4
10^5
10^6
10^7
10^8
10^9
Data Rate (bps)

Performance Measures

- **Latency**: How long does it take to get a particular task done?
  - Also called execution time or running time
  - Usually measured in time (e.g., microseconds)
- **Throughput**: How many tasks can you perform in a unit of time?
  - Also related to bandwidth (communication channels, storage)
  - Usually measured in units per time (e.g., megabytes/ second)





Clock Speed

- Operations performed by a processor, such as fetching an instruction, decoding the instruction, performing an arithmetic operation, and so on, are governed by a system clock.
- Typically, all operations begin with the pulse of the clock.
- Thus, at the most fundamental level, the speed of a processor is dictated by the pulse frequency produced by the clock, measured in cycles per second, or Hertz (Hz).
- For example, a 1-GHz processor receives 1 billion pulses per second.
- *The rate of pulses is known as the clock rate, or clock speed.*
- One increment, or pulse, of the clock is referred to as a clock cycle, or a clock tick.
- The time between pulses is the cycle time.

Cycles Per Instruction (CPI)

- A micro operation is an elementary hardware operation that can be performed during one CPU clock cycle.
- This corresponds to one micro-instruction in microprogrammed CPUs.
- Examples: register operations: shift, load, clear, increment, ALU operations: add , subtract, etc.
- Thus a single machine instruction may take one or more CPU cycles to complete termed as the Cycles Per Instruction (CPI).
- Average CPI of a program: The average CPI of all instructions executed in the program on a given CPU design.

Number of Threads	Instructions	Clocks	CPI
1	600	1200	2
2	300	450	1.5
3	200	200	1
4	150	75	0.5

Computer/CPU Performance

- The Processor Performance Equation

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

Weighted CPI

Where I_i represents number of times instruction of type *i* is executed in a program and CPI_i represents the average number of clock cycles for instruction of type i. This form can be used to express CPU time as

$$\text{CPU time} = \left(\sum_{i=1}^{n} (CPI_i * I_i) \right) / \text{clock rate}$$

Computer/CPU Performance

- Different instruction types having different CPIs

$$\text{CPU clock cycles} = \sum_{i=1}^{n} IC_i \times CPI_i$$

$$\text{CPU time} = \left(\sum_{i=1}^{n} IC_i \times CPI_i \right) \times \text{Clock cycle time}$$

Instruction Frequency & CPI

- Given a program with n types or classes of instructions with the following characteristics:

C_i = Count of instructions of type$_i$ $\quad i = 1, 2, \ldots, n$

CPI_i = Average cycles per instruction of type$_i$

F_i = Frequency or fraction of instruction type$_i$ executed

= C_i / total executed instruction count = C_i / I

Then:

$$\boxed{CPI = \sum_{i=1}^{n} (CPI_i \times F_i)}$$

Fraction of total execution time for instructions of type i

$$= \frac{CPI_i \times F_i}{CPI}$$

Instruction Type Frequency & CPI: Example

Program Profile or Executed Instructions Mix

Base Machine (Reg / Reg)

Op	Freq. F_i	CPI_i	$CPI_i \times F_i$	% Time $\frac{CPI_i \times F_i}{CPI}$
ALU	50%	.1	.5	23% = .5/2.2
Load	20%	5	1.0	45% = 1/2.2
Store	10%	3	.3	14% = .3/2.2
Branch	20%	2	.4	18% = .4/2.2

Typical Mix

Sum = 2.2

$$CPI = \sum_{i=1}^{n} (CPI_i \times F_i)$$

$$CPI = .5 \times 1 + .2 \times 5 + .1 \times 3 + .2 \times 2 = 2.2$$
$$= .5 + 1 + .3 + .4$$

Hardware-Oriented Metrics

Clock rate and IPC are often combined into various figures of merit:

- **MIPS** (Millions of Instructions Per Second) – pronounced "mips"
- **MOPS** (Millions of Operations Per Second) – pronounced "mops"
- **MFLOPS** (Millions of Floating-point Operations Per Second) – pronounced "megaflops" and sometimes written "megaFLOPS"

Replace first letter with **K** (kilo), **G** (giga), **T** (tera), **P** (peta), etc., as appropriate.

(or even **E** (exa), **Z** (zeta) ..)

MIPS Calculation

One alternative to time as the metric is MIPS, or million instructions per second. For a given program, MIPS is simply

$$\text{MIPS} = \frac{\text{Instruction count}}{\text{Execution time} * 10^6} = \frac{\text{Clock rate}}{\text{CPI} * 10^6}$$

Misleading Performance Measurement

Clock rate: 500MHZ

	Instruction class		
	A	B	C
CPI	1	2	3

	Instruction counts (in billions) for each instruction class		
Code from	A	B	C
Compiler 1	5	1	1
Compiler 2	10	1	1

- Execution time=$\Sigma(CPI_i \cdot I_i)$/clock rate

  Execution time$_1$= $((1*5+2*1+3*1)*10^9)/(500*10^6)$=20s

  Execution time$_2$= $((1*10+2*1+3*1)*10^9)/(500*10^6)$=30s

- MIPS=instruction count/(execution time*10^6)

  MIPS$_1$= $((5+1+1)*10^9)/(20*10^6)$=350

  MIPS$_2$= $((10+1+1)*10^9)/(30*10^6)$=400

Problems with Hardware-Oriented Metrics

- Processors with different ISAs may require a different number of instructions to perform the same task, so MIPS hard to compare

  - MOPS and MFLOPS are a somewhat better measure

  - How do you count floating-point divides?

- Vendors usually report "peak" rates